

TDMA Schedule Planner and Optimizer

Planning collision-free time slots for radios,
and loading the plan into the EMANE network emulator

Aadya Jain

Register No. RA2411003010085

Wireless Developer Internship assignment
Vaan Megam Networks Pvt. Ltd.

Summary

This report describes a program that decides when each radio in a network is allowed to transmit, so that no two signals ever clash at a receiver. All radios share one frequency, so time is split into short slots and each radio gets its own slot. Radios that are far enough apart can safely share a slot, which keeps the schedule short. I model the radios as a graph and solve the problem as a graph colouring task, where each colour stands for a time slot. The program tries three well-known methods, keeps the best result, tries to improve it further, and then checks it against the smallest number of slots that is possible in theory. For the 16-radio grid used in the assignment, it finds a schedule of 9 slots, and I show that no valid schedule can use fewer. On 200 random networks of up to 150 radios, it always reached that theoretical minimum. In the second part, the schedule is loaded into the EMANE network emulator. With my schedule, EMANE recorded no collisions at all, while a simpler schedule that ignores hidden terminals lost over 60% of its packets to collisions.

1 Introduction

1.1 The problem

In a TDMA (Time Division Multiple Access) network, all radios use the same frequency and take turns. Time is divided into short slots, and a fixed group of slots, called a frame, repeats again and again. Each radio may only transmit in its own slot. The task is to write a program that reads the positions of 16 radios, gives each radio a slot, and prints the result as a list and as a table of slots against radios.

Two situations must be avoided:

- *Direct clash.* Two radios that can hear each other transmit at the same time.
- *Hidden terminal.* Two radios cannot hear each other, but they have a neighbour in common. If both transmit in the same slot, the shared neighbour receives both signals at once and cannot understand either of them. The two senders have no way of noticing this, which is why it is called the hidden terminal problem.

Radios that are three or more hops apart can safely use the same slot, because no single radio can hear both of them. This is called spatial reuse, and it is the main way to keep the frame short.

A short frame matters because each radio gets one turn per frame. With 1 ms slots, a 9-slot frame lets each radio transmit about 111 times per second. A 16-slot frame, with one slot per radio and no reuse, allows only about 62.

1.2 Assumptions

- Two radios can hear each other if they are 500 m apart or less. If A can hear B, then B can hear A.
- The radios do not move while the schedule is in use.

- Each radio needs one transmit slot per frame and listens during all other slots.
- The brief gives only a few example positions (Node_01 at (0, 0), Node_02 at (300, 0) and Node_16 at (900, 900)). These fit a 4×4 grid with 300 m between neighbours, so I use that grid as the main test case.

2 How the problem is modelled

I represent the network as a graph. Each radio is a point (a node), and a line (an edge) joins two radios that are within range. I call this the *radio graph*.

Giving slots to radios is then the same as colouring the nodes of a graph: each slot is a colour, and certain pairs of nodes must get different colours. Here the rule is that two radios must get different slots if they are one hop or two hops apart:

$$\text{slot}(A) \neq \text{slot}(B) \text{ whenever } A \text{ and } B \text{ are 1 or 2 hops apart.}$$

This is known as *distance-2 colouring*. To make it easier to solve, I build a second graph, the *conflict graph*, which joins every pair of radios that must not share a slot. In graph theory this is called the square of the radio graph. Once the conflict graph exists, the problem becomes an ordinary graph colouring problem, and many standard methods can be used on it directly.

Finding the smallest possible number of colours is a very hard problem. No known method can always find the best answer quickly for large networks [4]. For this reason, the brief asks for heuristics, which are practical rules that give good answers fast without a guarantee of being perfect. The same model is widely used for scheduling in radio networks [5].

3 Design of the scheduler

The scheduler is written in Python. It uses the NetworkX library [6] to store the graphs and matplotlib to draw the figures. The code is split into small files so that each part can be tested on its own.

File	What it does
<code>topology.py</code>	Reads the input and builds the radio graph and the conflict graph
<code>coloring.py</code>	The colouring methods and the steps that improve the result
<code>analysis.py</code>	Minimum slot count, safety check, and slot sharing statistics
<code>planner.py</code>	Runs all the steps in order
<code>report.py, plot.py</code>	Text report, JSON output, and figures
<code>schedule.py</code>	The command line program

Table 1: Code structure.

3.1 Building the conflict graph

First, the program measures the distance between every pair of radios and joins the pairs that are 500 m apart or less. For 16 radios this is only 120 pairs, so a simple check of every pair is fast enough.

Next it builds the conflict graph. NetworkX can do this in one call, but I wrote it out step by step because it follows the hidden terminal idea directly. For every radio B:

- join B to each of its neighbours, since they would clash directly, and
- join every pair of B's neighbours to each other, since those two would clash at B.

Any two radios that are two hops apart must have a neighbour in common, so this covers every case.

3.2 Giving out slots

All three methods use the same basic step, called greedy colouring. The radios are visited one at a time, and each one gets the lowest slot number that none of its conflicting radios already has. This is fast, but the result depends a lot on the order in which the radios are visited. The three methods differ only in how they choose that order:

- *Largest first*. Radios with the most conflicts go first. These are the hardest to place, so they are handled while many slots are still free.
- *Smallest last* [2]. The radio with the fewest conflicts is set aside again and again until none are left, and then the radios are given slots in the reverse order. This means each radio, when its turn comes, has only a few neighbours that already have a slot, so it rarely needs a new one.
- *DSATUR* [1]. At each step, the next radio is the one whose neighbours already use the most different slots. This radio has the fewest choices left, so it is handled before it runs out of options.

All three methods run on every input and the best result is kept. Each takes only a few milliseconds, so running all of them costs almost nothing.

3.3 Improving the result

The best result is then passed to a method called iterated greedy [3]. It groups the radios by the slot they currently have, changes the order of the groups, and runs greedy colouring again in that new order. Radios in the same group can always share a slot again, so this step never makes the schedule longer. Sometimes one group gets absorbed into the others, which saves a slot. I repeat this 300 times, using a fixed random seed so that the results can be reproduced.

If the schedule is still longer than the theoretical minimum (Section 3.4), the program runs an exact search. It tries, in an organised way, to give out slots using one slot fewer. If it finds a way, the schedule gets shorter. If it proves there is no way, the current schedule is the best possible. This search can be slow on large networks, so it has a 5-second time limit. If the time runs out, the program keeps the best result it already has.

3.4 The theoretical minimum

A heuristic gives an answer, but it does not say how good that answer is. To measure this, the program works out the smallest number of slots that any valid schedule could use, in two ways:

- *Busiest radio*. Take the radio with the most neighbours. That radio and all of its neighbours must each have a different slot, because the neighbours are all two hops apart through it. So the schedule needs at least (number of neighbours + 1) slots.
- *Largest clash group*. Find the largest group of radios in which every pair clashes. Each member of that group needs its own slot. NetworkX can find this group quickly for networks of this size. This bound is always at least as strong as the first one.

If the schedule uses exactly this many slots, or the exact search has shown that fewer is impossible, the report says the schedule is optimal.

3.5 Safety check

The final schedule is checked by separate code that does not use the conflict graph at all. It goes back to the radio graph, counts the hops between every pair of radios that share a slot, and reports any pair that is only one or two hops apart. If it finds a problem, the program prints it and ends with an error code. I added this because a mistake in building the conflict graph would otherwise go unnoticed: the schedule would look correct for the wrong graph.

3.6 The command line program

The radio positions are given as a JSON string in the format shown in the brief, or read from a file with `--file`. Other options set the radio range, the number of improvement rounds, the random seed, and the time limit for the exact search. The program can also save a picture of the network (`--plot`) and a JSON copy of the schedule for Part 2 (`--json-out`). The text output follows the sample in the brief, and also shows the theoretical minimum, a comparison of the three methods, and which radios share each slot.

4 Results

4.1 The 4×4 grid

With 300 m between neighbours and a 500 m range, each radio can hear the radios next to it horizontally, vertically and diagonally (the diagonal distance is 424 m). This gives 42 links and 90 pairs of radios that must not share a slot. The radios in the middle of the grid have 8 neighbours each, so the minimum is 9 slots. All three methods found a 9-slot schedule straight away, so the schedule is optimal. It is shown in Figure 1, and the slot table is given below.

Slot \ Node	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16
Slot 00	1	0	0	1	0	0	0	0	0	0	0	0	1	0	0	1
Slot 01	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0
Slot 02	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0
Slot 03	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0
Slot 04	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
Slot 05	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
Slot 06	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0
Slot 07	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
Slot 08	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0

Twelve of the 16 radios share their slot with at least one other radio. The four corner radios all use slot 0, and each pair of corners is three hops apart, which is the closest distance at which sharing is allowed. The four radios in the centre each need their own slot, because each of them is within two hops of every other radio in the grid.

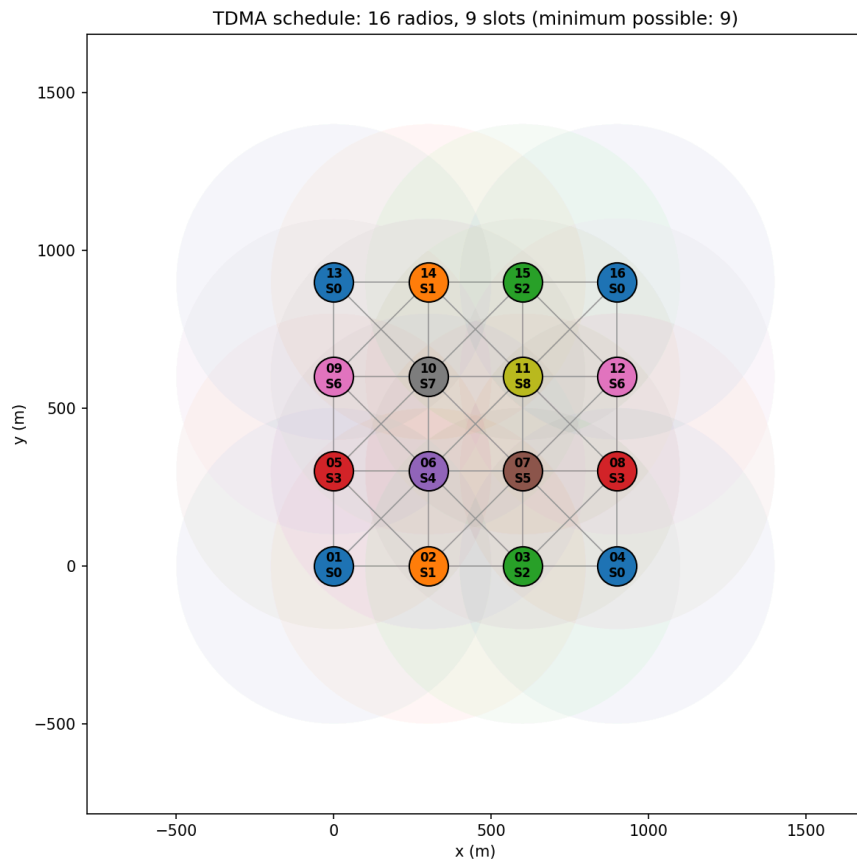


Figure 1: Slot plan for the 4×4 grid. Each circle shows the radio number and its slot. Grey lines are radio links, and the shaded areas show the 500 m range.

4.2 Other layouts

I also tested three other 16-radio layouts that are less regular than the grid.

Layout	Links	Clashing pairs	Slots used	Minimum	Radios per slot
4×4 grid, 300 m apart	42	90	9	9	1.78
Straight line, 350 m apart	15	29	3	3	5.33
Two groups 3 km apart	45	56	8	8	2.00
Random, 1.5 km square	27	48	6	6	2.67

Table 2: Results for the four example layouts. All four schedules are optimal.

The straight line shows spatial reuse most clearly. The slots repeat as 0, 1, 2, 0, 1, 2, ..., so 16 radios need only 3 slots. In the two-group layout, the groups are too far apart to disturb each other, so the larger group decides the frame length and the other group reuses the same slots.

4.3 Comparison with a simpler schedule

To show why the two-hop rule matters, I also made a simpler schedule for the grid that only keeps direct neighbours apart and ignores hidden terminals. It needs only 4 slots, which looks much better at first. However, the safety check finds 24 pairs of radios that share a slot and also share a neighbour (Figure 2). At each of those shared neighbours, packets would be lost whenever both senders are active.

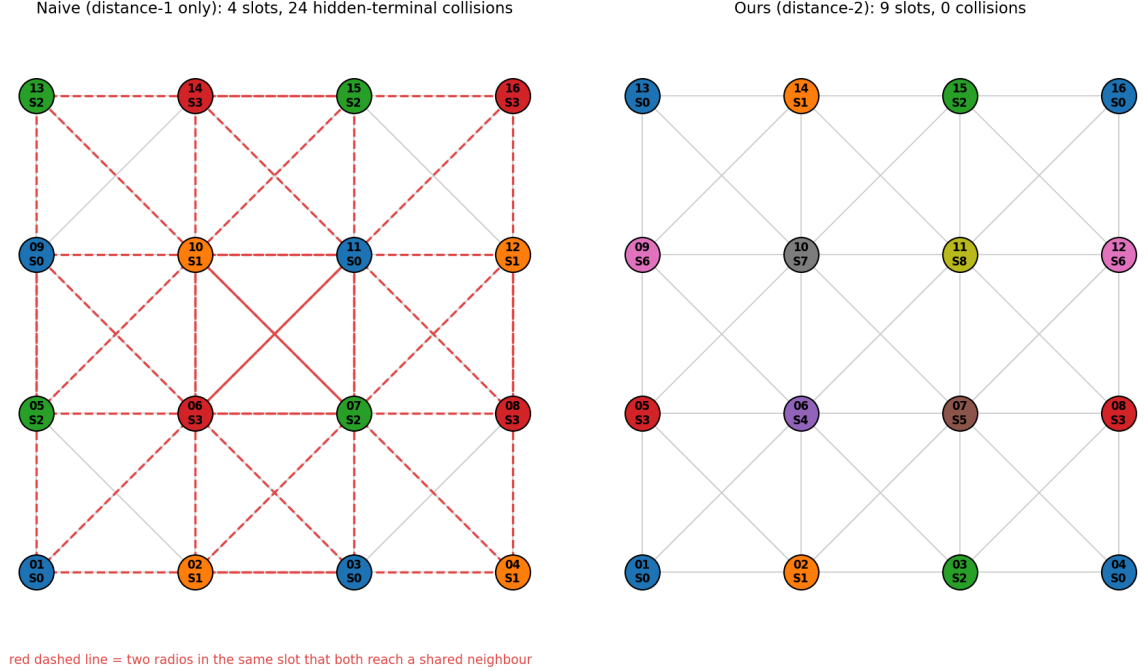


Figure 2: The simpler one-hop schedule (left) and the two-hop schedule (right) on the same grid. Dashed red lines join radios that share a slot and a neighbour.

4.4 Testing on larger networks

With only 16 radios, most sensible methods do well, so I also tested larger networks. I created 40 random networks for each size of 16, 32, 64, 100 and 150 radios. The area grew with the number of radios, so the networks stayed equally crowded. For each network I recorded how many slots each method used above the theoretical minimum.

Radios	Largest first	Smallest last	DSATUR	Final result	Optimal	Average time
16	0.00	0.00	0.00	0.00	40 / 40	5 ms
32	0.05	0.00	0.00	0.00	40 / 40	12 ms
64	0.17	0.00	0.03	0.00	40 / 40	26 ms
100	0.25	0.00	0.00	0.00	40 / 40	54 ms
150	0.17	0.00	0.05	0.00	40 / 40	125 ms

Table 3: Average number of extra slots above the minimum, over 40 random networks per size. "Final result" is the output of the full program. The time covers all steps.

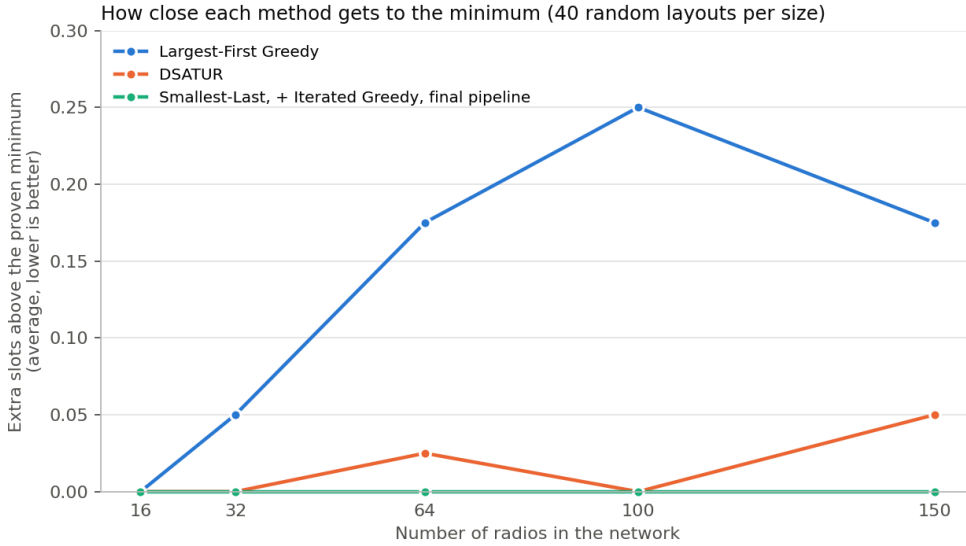


Figure 3: The results from Table 3 plotted against network size.

Largest first was the weakest method and sometimes used one extra slot. Smallest last never did, in any of the 200 networks. I think this is because the networks are based on distance: in any group of radios, some are near the edge and have only a few conflicts, and smallest last makes good use of exactly that. The full program reached the minimum every time. On these networks, the improvement step and the exact search did not change any result. Their real value here was confirming that each result was the best possible. I also tried networks where radios were bunched into a few clusters, and the results were the same.

4.5 A note on the sample output in the brief

While testing the safety check, I ran the sample schedule from the brief (5 slots) on the 300 m grid. It found 21 hidden terminal clashes. For example, Node_01 and Node_03 are both in slot

0, and both can reach Node_02. Since the minimum for this grid is 9 slots, no 5-slot schedule can be correct for it. The sample was probably meant only to show the output format, or was made for different positions, but it was a useful test that the safety check catches real mistakes.

5 EMANE integration (Part 2)

EMANE is a network emulator: it imitates real radios in software, so a radio network can be tested on one computer. The goal of Part 2 is to load the schedule into EMANE's TDMA radio model and show that traffic between the emulated radios follows it.

5.1 Setup

The brief allows either one Linux machine or one container. I used a single Docker container, and gave each radio its own Linux network namespace. A namespace is a separate copy of the network settings, so each radio behaves like its own small computer, without needing 16 containers. EMANE's ready-made packages only exist for Intel and AMD processors, and my laptop has an ARM processor, so the Dockerfile builds EMANE from its source code [7] on Ubuntu 22.04.

5.2 Radio settings

Each emulated radio uses EMANE's TDMA model. This model does not create a schedule itself. It waits to receive one, then transmits only in its own slots and listens in all the others. This fits the design well: my program decides the schedule, and EMANE follows it. The main settings are shown below.

Setting	Value	Reason
Slot length	1 ms	Required by the brief
Slots per frame	From my program (9 for the grid)	The frame length is the program's output
Frequency	2.4 GHz, one channel	All radios share one frequency
Data rate	10 Mbit/s	1,250 bytes per slot, enough for one ping packet
Who can hear whom	Set from the 500 m rule	Matches Part 1 exactly
Overlapping signals	Counted as interference	So that clashes really cause packet loss
Packet bundling	On, up to 50% of a slot	Lets busy radios send several packets per turn, with spare time left in the slot

Table 4: EMANE settings.

I did not want EMANE to work out by itself who can hear whom, because its idea of "in range" might be slightly different from mine. Instead, my program tells EMANE directly how much each signal weakens between each pair of radios (the path loss). Radios within 500 m get a strong signal (90 dB loss), and all others get a signal so weak that it cannot be heard (200 dB

loss). I also set EMANE to add up signals that arrive at the same time, so that a hidden terminal clash really causes packet loss in the emulation.

5.3 Converting the schedule

A converter script, `emane/bridge.py`, reads the schedule saved by my program and writes the files EMANE needs: the schedule itself, one settings file per radio, and the path loss table. Node_01 becomes EMANE radio 1 with address 10.100.0.1, and so on. Slot sharing appears directly in the schedule file, where one slot can list several radios:

```
<emane-tdma-schedule>
  <structure frames="1" slots="9" slotoverhead="0" slotduration="1000" bandwidth="1M"/>
  <multiframe frequency="2.4G" power="0" class="0" datarate="10M">
    <frame index="0">
      <slot index="0" nodes="1,4,13,16"/>
      <slot index="1" nodes="2,14"/>
      <slot index="2" nodes="3,15"/>
      ...
      <slot index="8" nodes="11"/>
    </frame>
  </multiframe>
</emane-tdma-schedule>
```

The converter refuses to run if my program did not mark the schedule as safe. With the `--naive` option, it writes the simpler one-hop schedule from Section 4.3 instead, which is used for comparison. A test converts a schedule into the EMANE format, reads it back, and checks that nothing has changed.

5.4 Test plan

The script `emane/run_demo.sh` runs inside the container and does the following, first with my schedule and then with the simpler one:

1. Create the schedule and convert it to the EMANE format.
2. Start 16 emulated radios, each in its own namespace.
3. Send EMANE the path loss table and then the schedule.
4. Make every radio send pings to all of its neighbours at the same time (50 packets each, 20 per second), so the channel is busy, and record how many packets are lost on each link.
5. Read each radio's slot statistics to see which slots it actually transmitted in.

If EMANE follows the schedule, my schedule should show little or no packet loss, and each radio should only transmit in its own slot. The simpler schedule should lose packets on links where the receiver has a hidden terminal.

5.5 Results

The emulation was run on my laptop, inside one Docker container, with 16 emulated radios on the 4×4 grid. Each radio sent 50 pings to each of its neighbours at the same time, which gives 84 links in total. EMANE keeps a count of every dropped packet together with the reason, and I used these counts to separate real collisions from other problems.

First, EMANE followed the schedule exactly. Each radio's slot table showed that it transmitted only in the slot my program gave it (for example, Node_01 only in slot 0 and Node_06 only in slot 4) and listened in all other slots.

Slot length	Schedule	Average loss	Links with no loss	Collision drops	Timing drops
1 ms	Two-hop (mine)	9.9%	37 / 84	0	166,082
1 ms	One-hop (simple)	63.8%	0 / 84	1,078	1,053,460
5 ms	Two-hop (mine)	0.0%	84 / 84	0	0
5 ms	One-hop (simple)	61.4%	0 / 84	20,688	0

Table 5: EMANE results. Drop counts are in bytes, added up over all 16 radios. Collision drops are packets lost because two signals overlapped at the receiver. Timing drops are packets that reached the receiver after their slot had already ended.

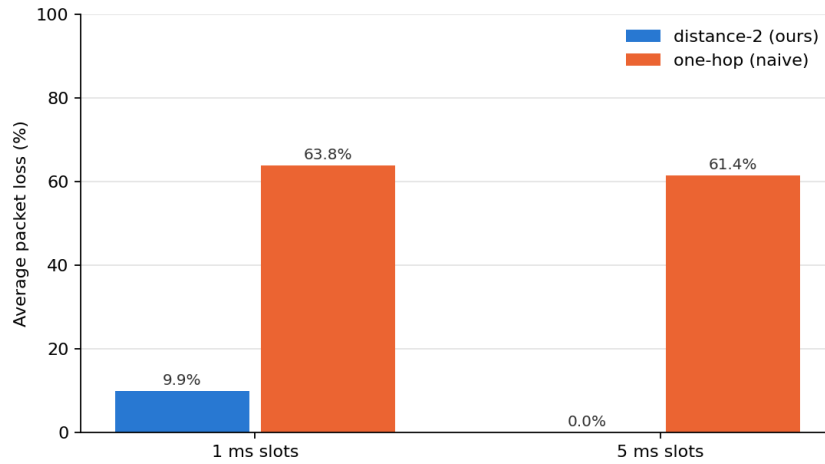


Figure 4: Average packet loss in EMANE for the two schedules.

With 1 ms slots, as asked in the brief, my schedule had no collision drops at all, while the one-hop schedule had collisions, as expected from Section 4.3. However, my schedule still lost 9.9% of its packets. The drop counts showed that every one of these losses was a timing drop: the packet arrived after its slot had ended. The slot tables showed packets arriving between a quarter and three quarters of a slot late. The cause is the test machine, not the schedule. Sixteen emulated radios share two CPU cores in a small virtual machine, and passing a packet between them takes close to a millisecond, which is about the length of a whole slot.

To confirm this, I ran the same test again with 5 ms slots, which gives each packet enough time to arrive within its slot. Nothing else was changed. My schedule then delivered every packet on all 84 links. The one-hop schedule still lost 61.4% of its packets, and this time every single one of its drops was a collision. Eight of its links lost all of their packets.

These results show that EMANE sends and blocks traffic strictly according to the schedule from Part 1, and that the two-hop rule is what prevents collisions. On real radio hardware, or on a faster machine, I would expect the 1 ms setup to behave like the 5 ms run here.

5.6 Problems I ran into

- The first EMANE build failed because my laptop ran out of disk space. After freeing space, it built from source without changes.
- EMANE expects the path to its reception curve file to be written as a `file://` address. I found this by comparing my settings with the example files that come with EMANE.
- With bundling switched off, each radio could send only one packet per turn, and busy radios filled their queues (over 5,000 queue overflow drops). With bundling up to 90% of a slot, long bundles ran past the end of the slot. A limit of 50% worked best.

6 Limitations and next steps

Range model. I treat the range as a perfect 500 m circle. Real signals fade gradually, so a radio just outside 500 m can still cause some interference. A better version would decide links from signal strength, using the same model as EMANE.

One slot per radio. Every radio gets exactly one slot, but in a real network some radios, such as relays, carry much more traffic than others. The program could be extended to give busy radios more than one slot.

Fixed positions. The schedule is made once, for radios that do not move. In a mobile network the links change over time. Making a completely new schedule each time would move most radios to new slots, so a better approach would update only the radios near the change.

Central planner. The brief asks for a central planner, but if it fails, the whole network is affected. Methods exist where radios agree on slots among themselves, and these would be the next step for a real mobile network.

7 Conclusion

Treating slot planning as graph colouring made it possible to use proven colouring methods directly. Running several methods and comparing the result with the theoretical minimum turned out to be more useful than relying on one clever method, because it shows when the answer cannot be improved. For the 16-radio grid, the program produces a 9-slot schedule, which is the best possible, and it reached the minimum on all 200 random networks tested. In Part 2, EMANE followed the schedule exactly. With 5 ms slots my schedule delivered every packet, while the simpler one-hop schedule lost 61.4% of its packets to collisions. With 1 ms slots my schedule still had no collisions, and its remaining losses came from the speed of the test machine.

References

- [1] D. Brélaz, "New methods to color the vertices of a graph," *Communications of the ACM*, vol. 22, no. 4, pp. 251–256, 1979.
- [2] D. W. Matula and L. L. Beck, "Smallest-last ordering and clustering and graph coloring algorithms," *Journal of the ACM*, vol. 30, no. 3, pp. 417–427, 1983.
- [3] J. C. Culberson, "Iterated greedy graph coloring and the difficulty landscape," Technical Report TR 92-07, University of Alberta, 1992.
- [4] S. T. McCormick, "Optimal approximation of sparse Hessians and its equivalence to a graph coloring problem," *Mathematical Programming*, vol. 26, pp. 153–171, 1983.
- [5] S. Ramanathan, "A unified framework and algorithm for channel assignment in wireless networks," *Wireless Networks*, vol. 5, pp. 81–94, 1999.
- [6] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conference*, 2008.
- [7] Adjacent Link LLC, EMANE: Extendable Mobile Ad-hoc Network Emulator, <https://github.com/adjacentlink/emane>, and TDMA radio model documentation, <https://emane.io/tdma-radio-model>.